



兰州工业学院

实习（实训）报告

（2024 —2025 学年第 1 学期）

实习（实训）名称 电子信息系统

综合实习

学 院 电子信息工程学院

专业班级 专升本电信 23

姓 名 姜 勇

学 号 ZB2305010116

指导教师 胡玫、吴记群

起止日期 2024. 12. 2--2024. 12. 13

目录

1. 绪论.....	1
1.1 项目背景.....	1
1.2 设计任务.....	1
2. 任务分析.....	2
2.1 功能分析.....	2
2.2 性能分析.....	2
3. 555 信号发生器原理与计算分析.....	2
3.1 555 定时器原理.....	2
3.1.1 多谐振荡器（方波发生器）.....	2
3.1.2 施密特触发器.....	2
3.1.3 单稳态触发器.....	2
3.2 频率计算公式.....	3
3.3 参数选择.....	3
4. 频率测量方案分析.....	3
4.1 测量原理.....	3
4.2 测量步骤.....	4
4.3 方案优势.....	4
5. 电路设计与 PCB 设计.....	4
5.1 系统框图.....	4
5.2 电路设计.....	5
5.2.1 单片机 AT89C51.....	5

5.2.2 晶振电路.....	5
5.2.3 复位电路.....	6
5.2.4 555 方波产生电路.....	7
5.2.5 最小系统电路.....	8
5.2.6 显示电路.....	9
5.2.7 串口通信电路.....	10
5.3 PCB 设计.....	12
6. 系统软件设计.....	12
6.1 开发工具.....	13
6.2 程序逻辑.....	13
6.2.1 主函数.....	13
6.2.2 频率测量.....	14
6.2.3 占空比测量.....	15
6.2.4 串口通信.....	16
7. 系统调试.....	16
7.1 仿真测试.....	17
7.2 系统测试与分析.....	18
实习（实训）总结.....	19
实习（实训）成绩评定表.....	20
附录 1 PCB 图.....	21
附录 2 仿真图.....	22
附录 3 源代码.....	23

实习（实训）时间	2024 年 12 月 2 日-2024 年 12 月 13 日
实习（实训）地点	DSP 应用技术实验室
实习（实训）目的 电子信息系统综合实习是电子信息工程专业的专业综合实践环节。通过典型的电子信息产品系统设计，要求学生应用电子信息学科的专业知识，掌握电子信息产品综合设计的相关技术，包括方案论证、单元电路设计、电路原理图绘制、微处理器最小系统与外设设计、软件设计和电路仿真。培养学生的实践能力、工程设计能力和综合能力，语言组织表达能力，报告撰写能力、提高分析问题解决实际问题的能力。实习过程中，培养学生团队协作、实事求是、刻苦钻研的科学精神，建立正确的科学观，求真务实，开拓进取，具有批判性思维，创新意识，学术诚信，使学生坚定致力于我国处理器应用系统设计与开发。	
实习（实训）日志	
日期：12 月 2 日 实习（实训）内容： 1. 绪论 1.1 项目背景 本次实训是电子信息产品综合设计实践的重要环节，目的是通过频率计的设计与实现，掌握电子系统开发的全流程，包括硬件设计、软件编程及仿真验证。通过设计一个单片机控制的频率计，学习如何使用单片机进行信号采集、频率测量和显示，从而提升设计和实现电子系统的能力。 1.2 设计任务 - 设计一个基于单片机控制的频率计，能够测量 TTL 信号的频率和占空比，并在 LCD 显示屏上实时显示测量结果。 测量范围： - 频率范围为 10Hz~200kHz，相对误差不超过 1%。 - 占空比范围为 0%~100%，测量精度为±1%。 测量速度：每秒测量频率和占空比，不低于 1 次/秒。	

- **显示分辨率：**频率分辨率为 0.1Hz，显示占空比的精度为 1%。

日期：12 月 3 日

实习（实训）内容：

2. 任务分析

2.1 功能分析

- 使用 555 定时器产生测试信号，包括频率和占空比可调的 TTL 信号。
- 单片机通过外部中断和定时器实现信号的频率和占空比测量。
- 测量结果通过 LCD 显示。

2.2 性能分析

- 系统需保证稳定性，测量误差和速度需满足需求。
- 易用性：硬件和软件的接口清晰，便于调试和验证。

3. 555 信号发生器原理与计算分析

3.1 555 定时器原理

555 定时器是一种多功能的集成电路，它可以被配置为单稳态触发器、多谐振荡器和施密特触发器等。在信号发生器中，555 定时器通常被用作多谐振荡器，产生方波信号。

3.1.1 多谐振荡器（方波发生器）

555 定时器与外部的电阻和电容一起，可以构成一个多谐振荡器。当电源接通时，电容通过电阻充电，当电容上的电压达到一定阈值时，555 定时器的输出状态会翻转，电容开始放电，如此循环，产生方波输出。

3.1.2 施密特触发器

555 定时器还可以组成施密特触发器，用于波形整形和滞后比较。施密特触发器有两个阈值电压，输入信号必须超过这些阈值才能改变输出状态。

3.1.3 单稳态触发器

555 定时器可以配置为单稳态触发器，它在接收到触发信号后输出一个固定时间的脉冲。

日期：12 月 4 日

实习（实训）内容：

3.2 频率计算公式

当用作多谐振荡器时，555 定时器的输出会在高电平和低电平之间切换，形成周期性的脉冲序列。这种模式下，电容器会交替充电和放电，从而控制输出波形的时间间隔。

对于一个多谐振荡器配置，555 定时器产生的矩形波频率 f 和占空比 Duty Cycle 可以根据以下公式计算：

$$\text{频率 (f): } f = \frac{1}{T} = \frac{1.44}{(R1 + 2R2)C}$$

其中 $R1$ 和 $R2$ 是外部连接的电阻值（欧姆）， C 是外部电容器的容量（法拉）。

$$\text{高电平时长 (t_High): } t_{\text{High}} = 0.693 \times (R1 + R2) \times C$$

$$\text{低电平时长 (t_Low): } t_{\text{Low}} = 0.693 \times R2 \times C$$

$$\text{占空比 (Duty Cycle): } \text{Duty Cycle} = \frac{t_{\text{High}}}{t_{\text{High}} + t_{\text{Low}}}$$

3.3 参数选择

- 设定频率范围 10Hz~200kHz，根据公式选择合适的电阻、电容组合。

日期：12 月 5 日

实习（实训）内容：

4. 频率测量方案分析

4.1 测量原理

- 频率测量利用单片机定时器功能，通过捕获信号周期内的脉冲数量来计算频率。定时器 0 用于计数信号脉冲，定时器 1 用于时间基准。
- 测量占空比时，分别捕获信号的高电平持续时间和完整周期时间，计算高电平占总周期的比例。

4.2 测量步骤

• 频率测量：

1. 使用定时器 1 作为时间基准，配置为定时模式，每 50ms 产生一次中断。
2. 定时器 0 配置为计数模式，记录信号的脉冲数。
3. 时间基准到达 20 次中断（即 1 秒）后，停止计数，计算频率：

$$f = \frac{\text{脉冲数}}{\text{计时周期}}$$

4. 显示结果时，将频率值分离为整数部分和小数部分，通过 LCD 显示。

• 占空比测量：

1. 利用信号的上升沿、下降沿及再次上升沿的捕获时间，计算高电平时间 (t_{High}) 和周期时间 (t_{Cycle})。
2. 计算占空比：

$$\text{占空比} = \frac{t_{\text{High}}}{t_{\text{Cycle}}} \times 100\%$$

4.3 方案优势

- **精度高：**定时器的高分辨率保证测量精度。
- **实时性好：**通过硬件中断响应实现快速捕获，减少软件延迟。
- **低计算复杂度：**将频率和占空比计算简化为基本的加法、乘法和除法，

减轻单片机的负担。

日期：12 月 6 日

实习（实训）内容：

5. 电路设计与 PCB 设计

5.1 系统框图

- **信号产生模块：**采用 555 定时器产生 TTL 方波信号，频率与占空比可调。
- **信号测量模块：**基于 C51 单片机，利用定时器和外部中断实现信号捕获与测量。
- **显示模块：**LCD1602 实时显示测量数据（频率、占空比）。

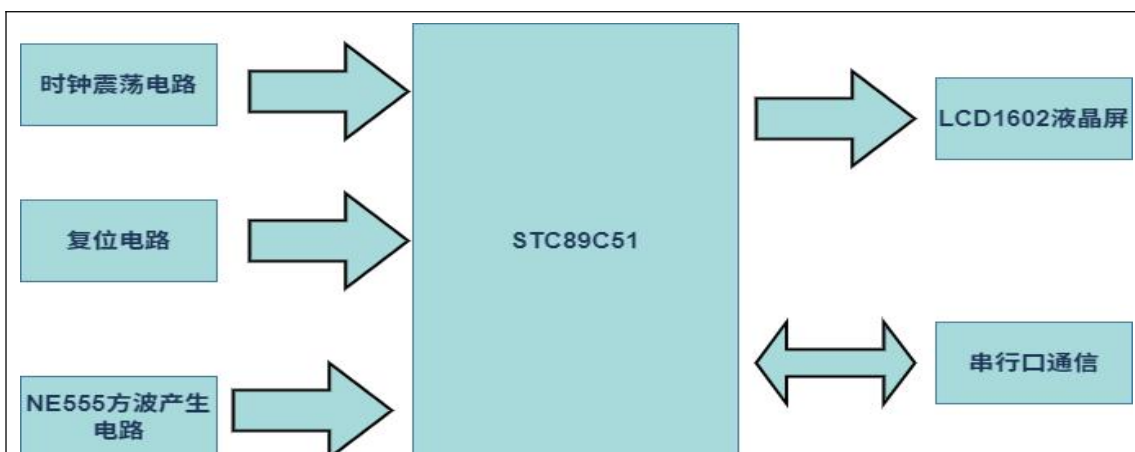


图 5-1 系统结构框图

日期：12 月 7 日

实习（实训）内容：

5.2 电路设计

5.2.1 单片机 AT89C51

硬件系统的核心处理器，是一种低功耗、高性能 CMOS8 位微控制器，具有 8K 在系统可编程 Flash 存储器，这也使得其为众多嵌入式控制应用系统提供高灵活、超有效的解决方案。

单片机的引脚排列图如图 5-2 所示：

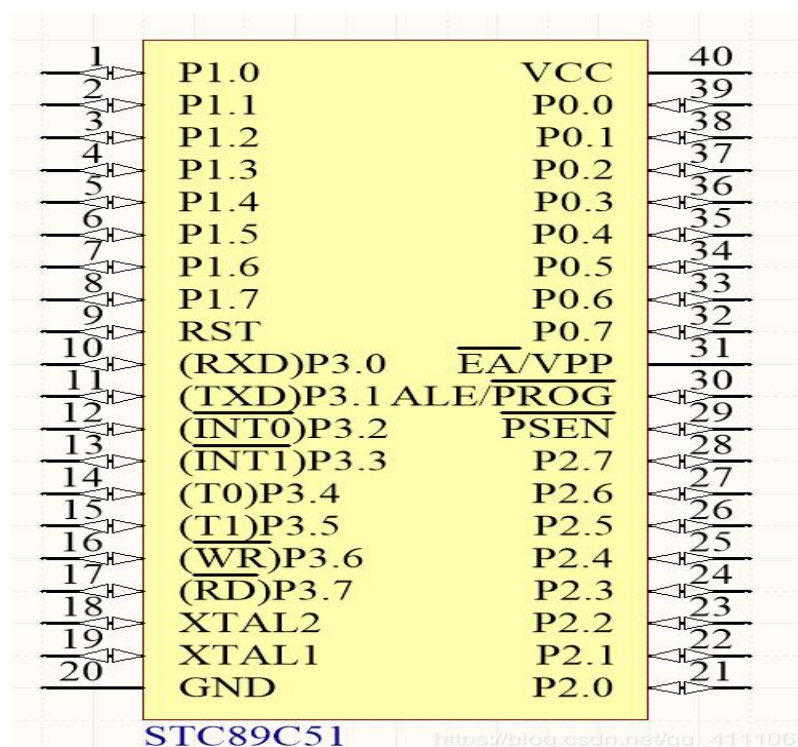


图 5-2 单片机引脚图

5.2.2 晶振电路

AT89C51 内部含有一个振荡器,可以作为 CPU 的时钟源,也可以由外部振荡器输入时钟信号,作为 CPU 的时钟源。高频率的时钟有利于程序更快的运行,也可以实现更高的信号采样率,从而实现更多的功能。但是对系统要求较高,而且功耗大,运行环境苛刻。考虑到单片机本身用在控制,并非高速信号采样处理,所以选取合适的频率即可。合适的频率的晶振对于选频信号强度准确度都有好处,这里采用的便是晶振频率 $F_{OSC}=12\text{MHz}$ 。

晶振电路如图 5-3 所示:

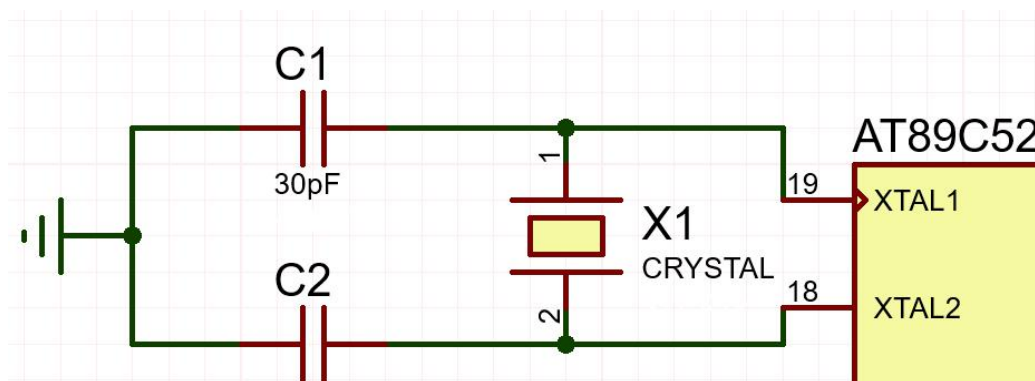


图 5-3 晶振电路原理图

5.2.3 复位电路

复位电路是维持单片机最小系统运行的基本模块。复位是单片机的初始化操作,只要给 RESET 引脚加上 2 个机器周期以上的高电平信号,就可使单片机复位。复位的主要功能是把 PC 初始化为 0000H,使单片机从 0000H 单元开始执行程序。除了进入系统的正常初始化之外,当由于程序运行出错或操作错误使系统处于死锁状态,为摆脱死锁状态,也需按复位键重新启动。经复位电路的好坏直接影响单片机系统工作的可靠性,而单片机的复位电路共有上电复位、按键复位和脉冲复位 3 种,在本次设计中采用按键复位,按键复位则是通过使复位端经电阻与 V_{CC} 电源接通来实现的。

复位电路的原理图如图 5-4 所示:

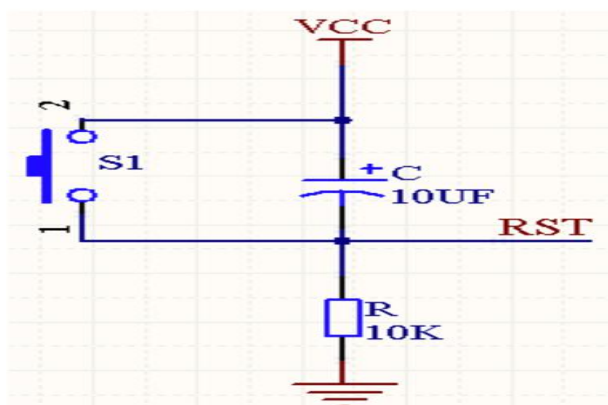


图 5-4 复位电路原理图

日期：12 月 8 日

实习（实训）内容：

5.2.4 555 方波产生电路

555 定时振荡器也就是通常所说的 555 定时器。555 定时器的用途非常广泛，可以生成各种类型的脉冲、创建时间延迟，也可以用于脉冲宽度调制（PWM），555 定时器最常见的用途是为电路产生时钟信号。

图 5-5 所示为 555 集成定时器的电路结构图，555 定时器电路由五个部分组成：

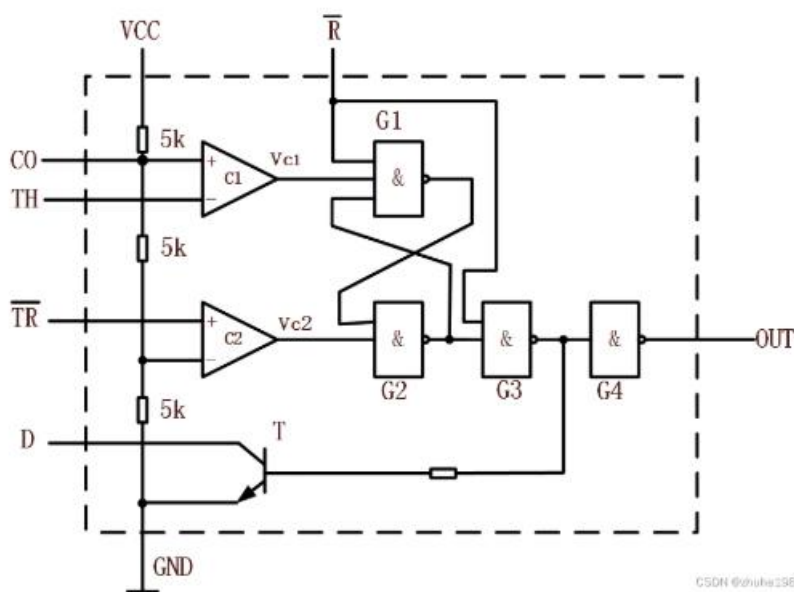


图 5-5 555 定时器结构图

(1) 由三个阻值为 5k 的电阻组成电阻分压器，为电压比较器 C1 和 C2 提供基准电压 V_{r1} 和 V_{r2} ；当 5 脚 C0 悬空时， $V_{r1}=(2/3)V_{cc}$ ， $V_{r2}=(1/3)V_{cc}$ 。C0 为控制电压输入端，可在此外加控制电压改变基准电压值。C0 端不用时，外接 0.01 μ F 的去耦电容，以消除干扰，保证基准电压不变。

(2) 集成运算放大器组成两个电压比较器 C1 和 C2，每个电压比较器两个输入端标有“+”号和“-”号：

当 $v_+ > v_-$ ，比较器输出 $v_o=1$ ；

当 $v_+ < v_-$ ，比较器输出 $v_o=0$ ；

(3) 基本 SR 触发器，其置 0 端(R 端)和置 1 端(S 端)均为低电平有效触发，

RS 的值取决于比较器的输出 ($R=V_{c1}$, $S=V_{c2}$)。

(4) 放电三极管 T, 相当于一个受控电子开关; 当输出 OUT 为 0 时, T 导通; 输出 OUT 为 1 时, T 截止。

(5) 输出缓冲器 G3 和 G4, 主要用于提高电路的带负载能力, 同时也隔离了负载对定时器的影响。

555 定时器方波产生电路的原理图如图 5-6 所示:

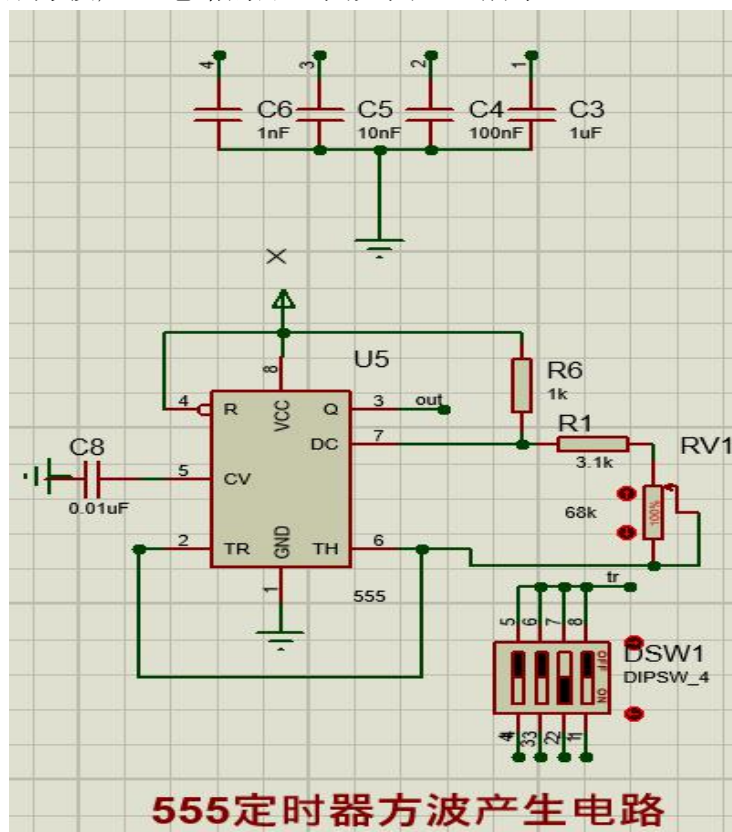


图 5-6 555 方波产生电路

日期: 12 月 9 日

实习 (实训) 内容:

5.2.5 最小系统电路

- 单片机型号: AT89C51。
- 信号输入:
 - 方波信号接单片机 P3.4 引脚, 用于捕获上升沿和下降沿。
 - 复位电路: 外接复位按钮和电容。
- 时钟电路: 外接晶振和匹配电容。

- 数据处理：定时器用于记录高电平和周期时间。

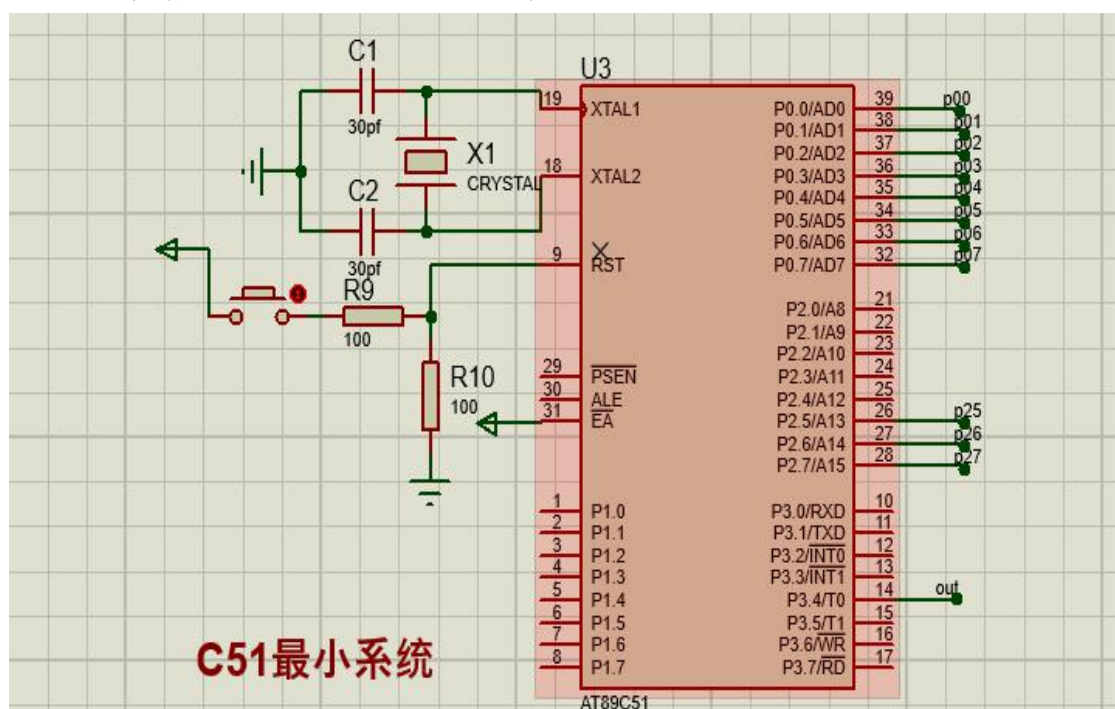


图 5-7 单片机最小系统

5.2.6 显示电路

LCD1602 液晶也叫 LCD1602 字符型液晶，它能显示 2 行字符信息，每行又能显示 16 个字符。它是一种专门用来显示字母、数字、符号的点阵型液晶模块。它是由若干个 5x7 或者 5x10 的点阵字符位组成，每个点阵字符位都可以用显示一个字符，每位之间有一个点距的间隔，每行之间也有间隔，起到了字符间距和行间距的作用，正因为如此，它不能很好的显示图片。

LCD1602 显示模块的工作原理

字符显示：LCD1602 使用字符点阵显示方式，通常每个字符由 5×8 点阵构成。通过控制这些点阵的亮灭状态，来显示字符或数字。

控制命令与数据输入：LCD1602 接受两类输入信号——命令和数据。命令用于控制显示器的状态（如清屏、光标控制等），而数据则是实际显示的内容。

LCD1602 显示接口的控制

在实际应用中，LCD1602 显示模块通过与单片机的连接来进行数据和命令的控制。对于 STC89C51 单片机，通常通过以下接口来控制 LCD1602 显示模块：

控制引脚：RS、RW 和 EN 引脚通过单片机的 I/O 口连接。RS 用于选择命

令寄存器或数据寄存器，RW 用于设置读写方向，EN 用于使能控制。

数据引脚：D0-D7 数据引脚通过 P0 口与单片机连接，负责传输命令或数据。

LCD1602 显示电路

LCD1602 电路与单片机连接时，常见的连接方式如下：

P0 口用于数据线连接（D0-D7）。

P2 口用于控制线连接（RS、RW、EN）。

电源和对比度调整：VSS 接地，VDD 接 5V，VO 接电位器调节对比度。

LCD1602 控制方式

初始化：初始化液晶显示模块，设置显示模式（4 位或 8 位），配置光标位置，设置光标是否闪烁等。

字符显示：通过将需要显示的字符的数据发送到 LCD，字符通过点阵显示。

命令控制：发送命令控制显示、清屏、返回光标等。

LCD1602 液晶显示电路如图 5-8 所示：

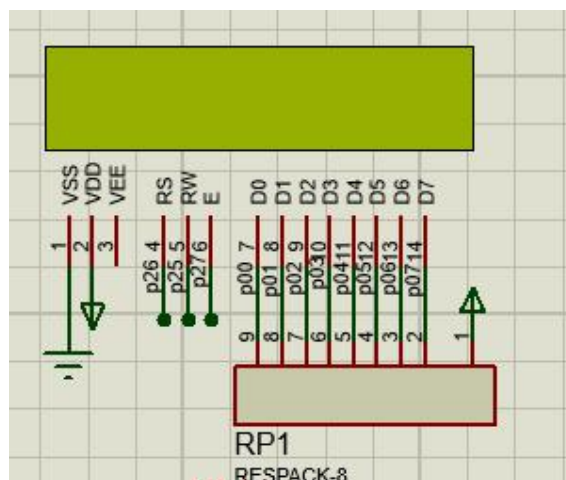


图 5-8 LCD1602 液晶显示

日期：12 月 10 日

实习（实训）内容：

5.2.7 串口通信电路

串口通信 (Serial Communication)，是指外设和计算机间通过数据信号线、地线等按位进行传输数据的一种通信方式，属于串行通信方式。

串口内部结构

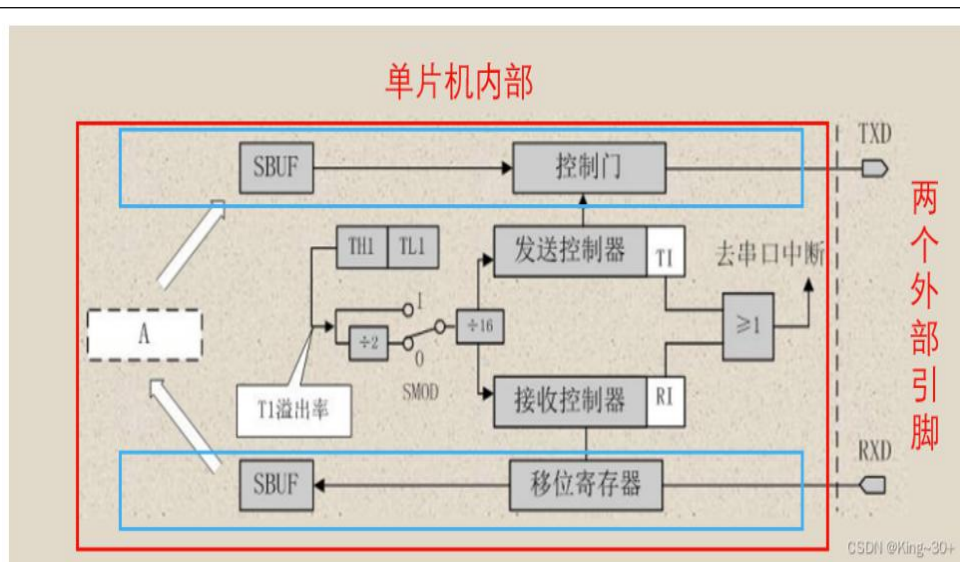


图 5-9 单片机内部串口结构

串行口进行数据的收发流程是：

串口数据的接收：外部数据通过 RXD 输入，先进入移位寄存器，在进入 SBUF（接受缓冲器）；当一帧数据全部进入 SBUF 后，串口发出中断请求，通知 CPU 读取数据，CPU 执行完读取 SBUF 中的数据，将其送入其他存储单元。同时接受端可接受下一帧数据。

串口数据的发送：CPU 将数据通过内部数据总线传送的移位寄存器，经过控制门逐位的把数据通过引脚 TXD 发送出去。在发送时，由于 CPU 主动进行操作，在发送外后，中断发出中断请求，可发送下一帧数据。

在上述的数据收发过程中，会牵扯一些相关的概念：

1、波特率发生器：主要由定时器 T0、T1 以及内部的一些控制开关和分频器组成。它为串口提供时钟信号，从而控制信号的收发。相关的寄存器有：TMOD、TCON、TL0、TH0、TL1、TH1 和 PCON；

2、串行数据缓冲寄存器 SBUF：用于接收和发送数据的缓存器；

3、串行口控制寄存器 TCON:用来控制串行通信接口的工作方式，并指示串行口的工作状态；

4、RXD：串行数据的输入引脚，对应单片机 I/O 口的 (P3.0)；

5、TXD：串行数据的输出引脚，对应单片机 I/O 口的 (P3.1)；

串口通信电路如图 5-10 所示。

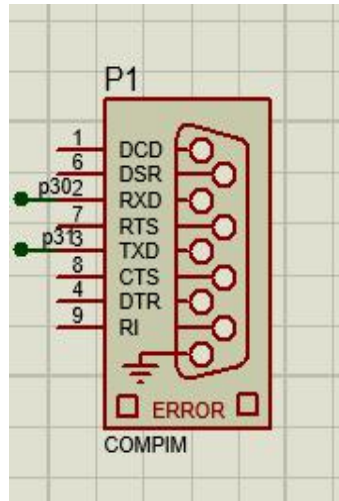


图 5-10 串口通信

日期：12 月 11 日

实习（实训）内容：

5.3 PCB 设计

- 短路径：信号线尽量短，减少干扰。
- 电源/地线：采用较宽的布线，地线形成完整环路。
- 区域分离：信号处理区、显示区、输入输出区分布明确，减少交叉干扰。

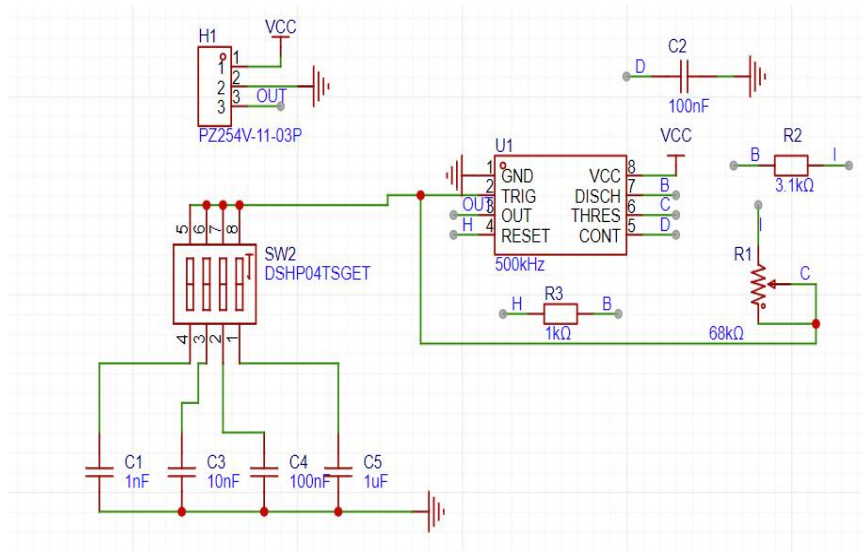


图 5-11 PCB 原理设计

日期：12 月 12 日

实习（实训）内容：

6. 系统软件设计

6.1 开发工具

- 使用 Keil5 编写代码。
- 使用 Proteus8 进行仿真测试。

6.2 程序逻辑

6.2.1 主函数

• 主函数是整个程序的核心，负责初始化硬件，启动频率和占空比测量，并在 LCD 显示结果。主函数流程如下：

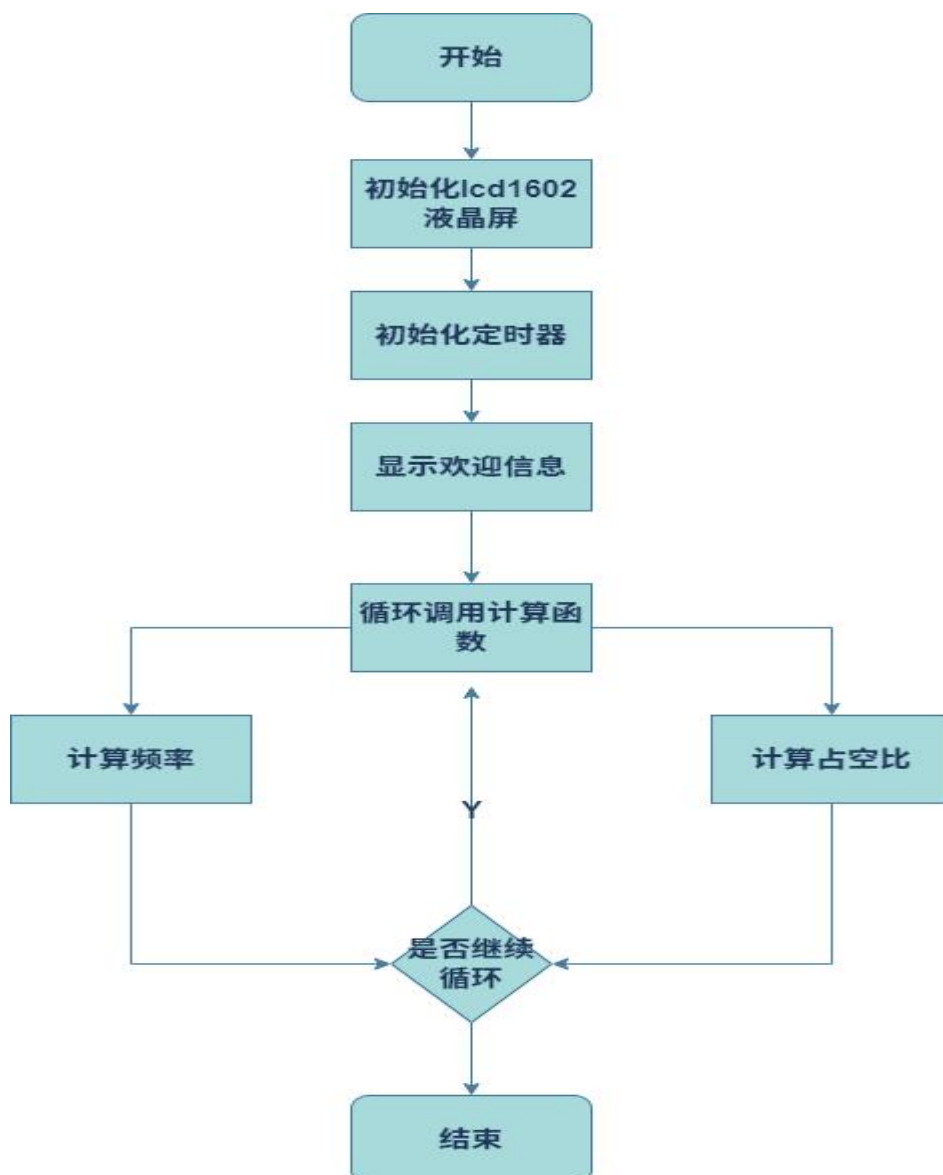


图 6-1 主函数

6.2.2 频率测量

频率测量流程主要依赖于单片机的定时器中断功能，捕获信号的周期时间并计算频率。测量流程如下：



图 6-2 频率测量

6.2.3 占空比测量

占空比的测量通过捕获信号的高电平时间和完整周期的时间差来完成。测量流程如下：



图 6-3 占空比测量

6.2.4 串口通信

上位机串口助手远程显示频率流程图如下：



图 6-4 串口通信

日期：12 月 13 日

实习（实训）内容：

7. 系统调试

7.1 仿真测试

以 Keil5 为软件调试平台，将程序编译生成 hex 文件，然后将 hex 文件通过 Proteus 下载到系统硬件电路进行仿真，然后观察测量结果。

对实际值为 2000HZ 的频率进行软件仿真调试，结果如图 7-1 所示。

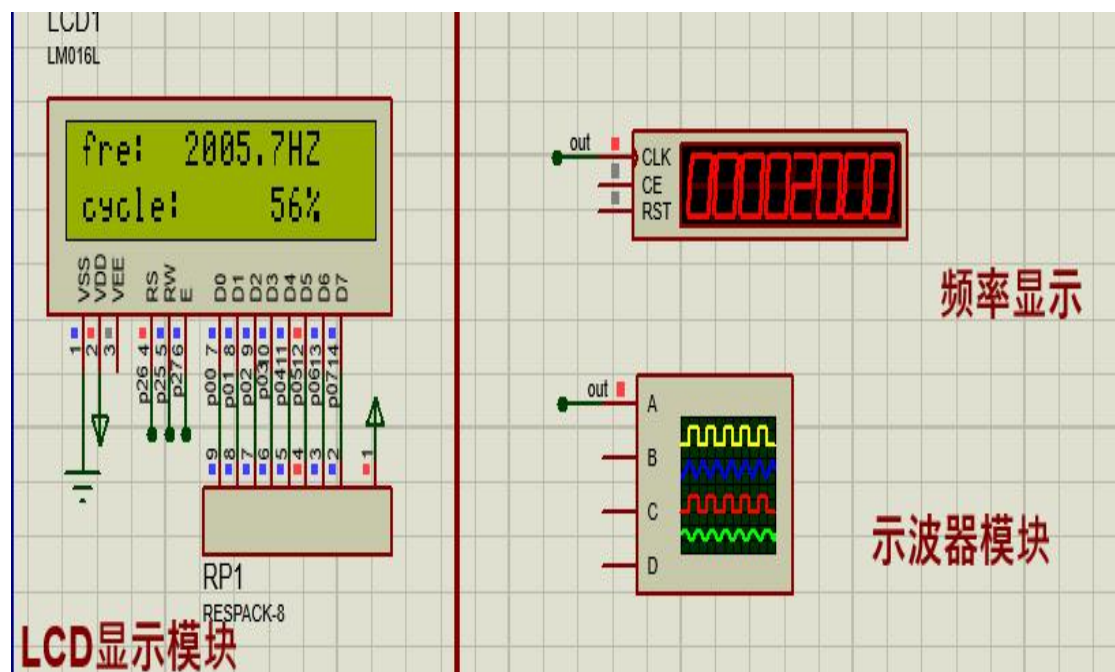


图 7-1 频率测量仿真结果图

由上图知，被测频率为 2000Hz，仿真结果为 2005.7Hz，误差为 0.2%，满足不超过 1%的误差要求。

对实际值为 200KHZ 的频率进行软件仿真调试，结果如图 7-2 所示。

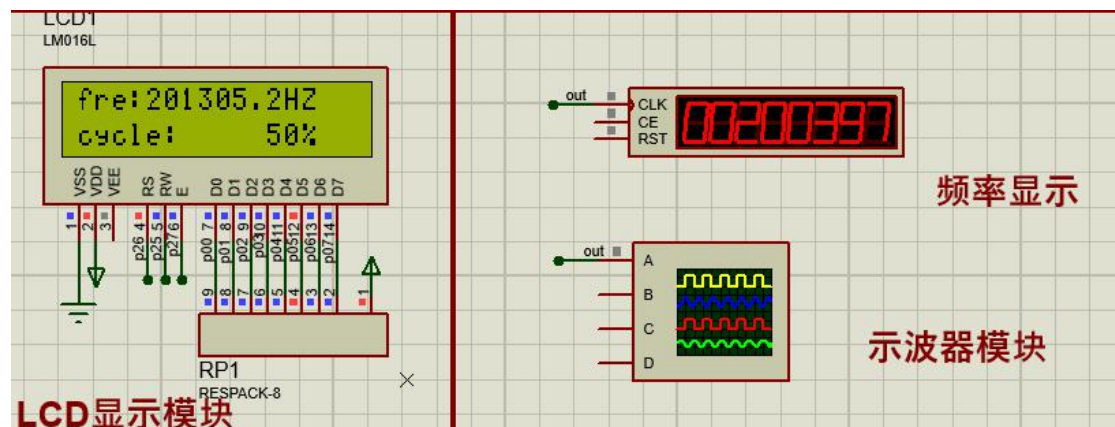


图 7-2 频率测量仿真结果图

由上图知，被测频率为 200397Hz，仿真结果为 201305.2Hz，误差为 0.4%，满足不超过 1%的误差要求。

7.2 系统测试与分析

测试时，分别对不同的频率进行测量，观察测量结果，通过对测量值与待测参数的标称值进行对比，计算相对误差，进而得出本系统的测量精度。

测试结果如表 7-1 所示。

表 7-1 频率测量数据

输入矩形波	理论值	测量值	误差（%）
10HZ	10HZ	10.0HZ	0
101HZ	101HZ	101.0HZ	0
205HZ	205HZ	206.0HZ	0.4
1007HZ	1007HZ	1010.6HZ	0.3
2051HZ	2051HZ	2057.3HZ	0.3
10052HZ	10052HZ	10087.0HZ	0.3
20457HZ	20457HZ	20523.1HZ	0.3
200000HZ	200000HZ	198293.7HZ	0.8

误差说明：

当测频时，启动定时计数器时，若从 T0（P3.4）输入矩形波刚好为高电平，而当 1s 定时到时刚好为高电平时，此时测得的频率值最准确。若启动定时计数器时，输入的矩形波刚好处于低电平，而当定时 1s 到时矩形波刚好要发生负跳变时，此时测得的频率误差最大。而且频率越大误差也越大。定时计数器的工作方式选择与初值的赋予不一定精准，容易引起误差。定时计数器的工作方式选择不同，最后的结果也会有所差异。工作方式 2 相比于工作方式 0 和工作方式 1 误差更小。其次，采用中断或查询的方式也会影响实验结果。采用查询方式的误差比采用中断误差更小。

实习（实训）总结

实习期间，我深入研究了 555 定时器的原理，并设计了基于 555 定时器的信号发生器，用于产生测试信号。同时，我学习了如何利用单片机的定时器和外部中断来实现信号的频率和占空比测量。在电路设计和 PCB 设计方面，我掌握了系统框图的绘制、电路图的详细设计，以及 PCB 布局的优化。此外，我还学习了如何使用 Keil5 进行软件开发和 Proteus8 进行仿真测试。

通过本次实习，我成功设计并实现了一个稳定的频率计系统。系统测试表明，该系统能够满足预定的性能指标，测量误差和速度均符合要求。在软件设计方面，我编写了完整的程序代码，并成功实现了频率和占空比的测量与显示功能。仿真测试结果表明，系统测量误差控制在 1%以内，满足了项目的设计要求。

这次实习极大地提升了我的实践技能和工程设计能力。我学会了如何将理论知识应用于实际问题中，并通过团队合作来解决复杂的工程问题。此外，我还提高了我的报告撰写能力和语言表达能力，这对于我未来的职业生涯无疑是一笔宝贵的财富。

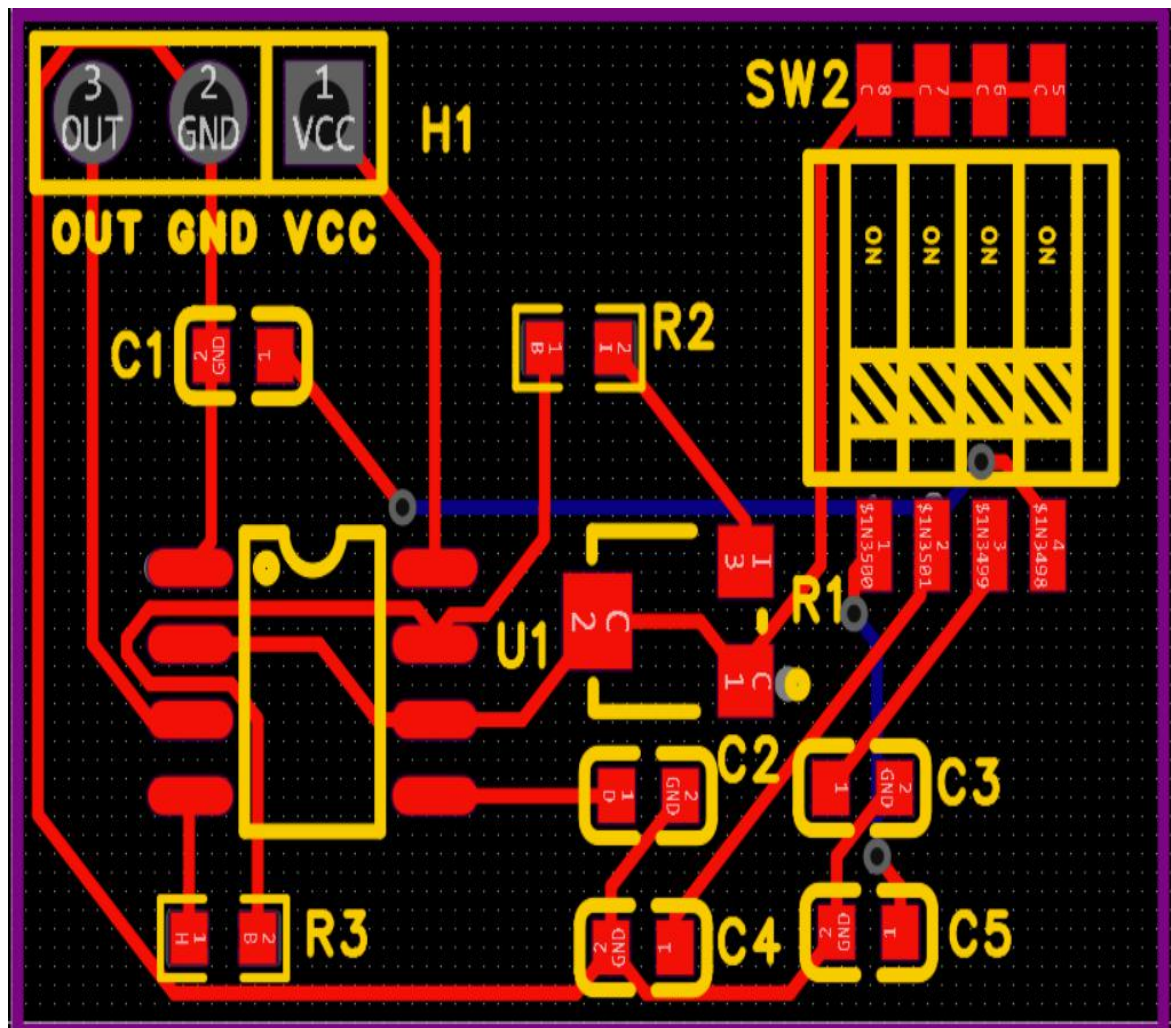
总的来说，这次电子信息系统综合实习是一次非常成功的学习经历。它不仅让我学到了宝贵的专业知识和技能，而且培养了我解决实际问题的能力。我相信，这些经验和技能将为我未来的学术研究和职业生涯打下坚实的基础。我期待将这些知识应用到更多的项目中，并继续在电子信息工程领域探索和进步。

实习（实训）成绩评定表

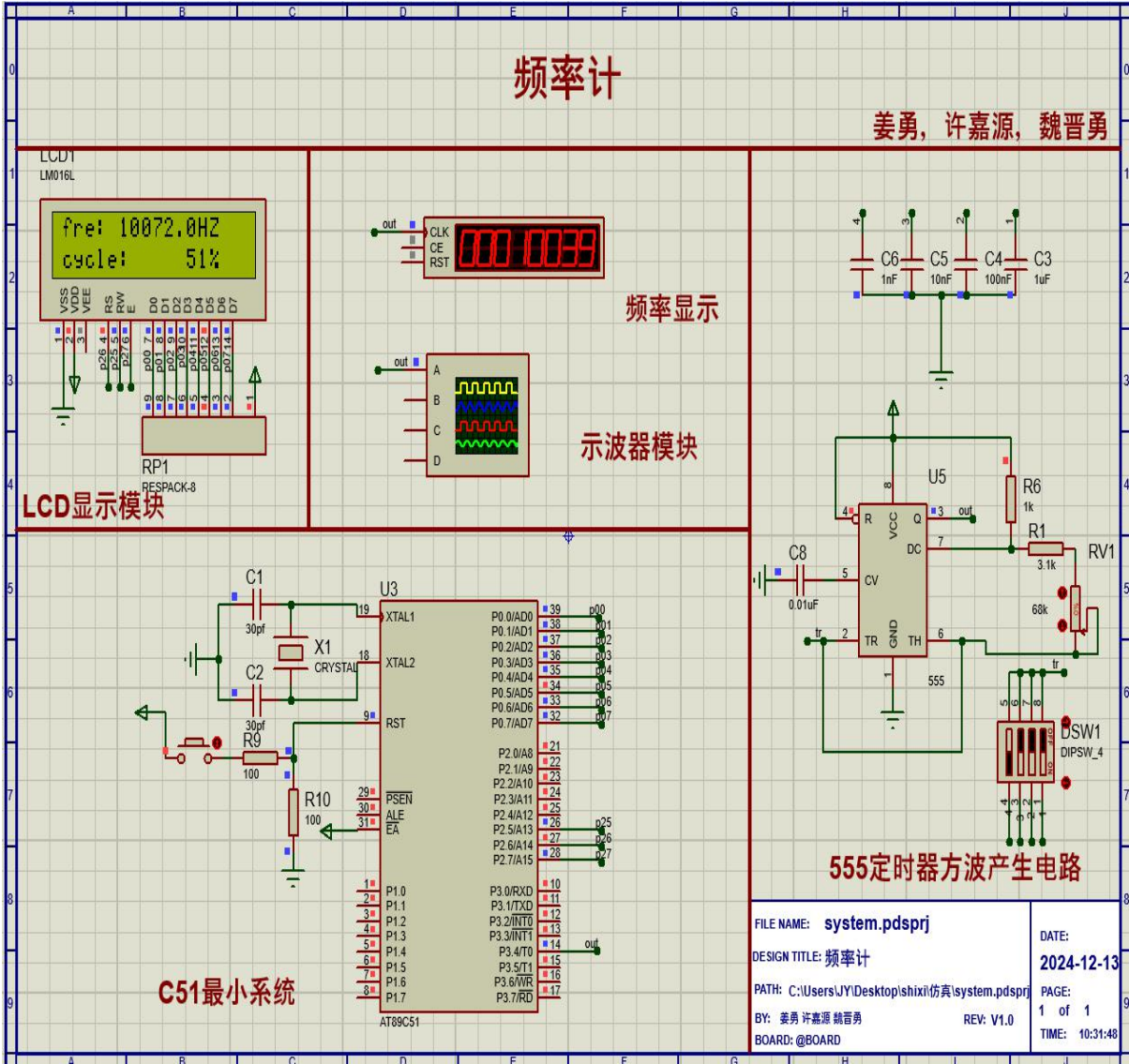
实习（实训）名称		电子信息系统综合实习	
序号	考核项目	分值分布	成绩
1	实习考核	30	
2	实习成果	40	
3	实习报告	30	
总分			
指导教师签字			

备注：考核项目及分值分布应依据实习（实训）大纲和实习（实训）内容确定。归档时应归入成绩评定原始记录。

附录 1 PCB 图



附录 2 仿真图



附录 3 源代码

```
#include "public.h"

// 延时函数，单位为毫秒
void delay(u16 ms)
{
    u8 m, n;
    for (m = ms; m > 0; m--) { // 外层循环控制总的延时时间（毫秒）
        for (n = 115; n > 0; n--); // 内层循环产生大约 1 毫秒的延时
    }
}

// 延时函数，单位为 10 微秒
void delay_10us(u16 ten_us)
{
    while (ten_us--); // 空循环，每次循环大约 10 微秒
}

#include "lcd1602.h"

// 向 LCD 写入命令
void lcd1602_write_cmd(u8 cmd)
{
    RS = 0; // 设置 RS 为 0 表示写入的是命令
    RW = 0; // 设置 RW 为 0 表示写操作
    EN = 0; // 禁用使能信号
    P0 = cmd; // 将命令写入 P0 端口
    delay(1); // 延时
    EN = 1; // 使能信号高电平触发
    delay(1); // 延时
    EN = 0; // 使能信号低电平结束
```

```

}

// 向 LCD 写入数据
void lcd1602_write_dat(u8 dat)
{
    RS = 1; // 设置 RS 为 1 表示写入的是数据
    RW = 0; // 设置 RW 为 0 表示写操作
    EN = 0; // 禁用使能信号
    P0 = dat; // 将数据写入 P0 端口
    delay(1); // 延时
    EN = 1; // 使能信号高电平触发
    delay(1); // 延时
    EN = 0; // 使能信号低电平结束
}

// 初始化 LCD
void lcd1602_init()
{
    lcd1602_write_cmd(0x38); // 设置 8 位数据接口，两行显示，5x7 点阵
    lcd1602_write_cmd(0x0c); // 显示开，光标关，不闪烁
    lcd1602_write_cmd(0x06); // 文字不动，地址自动加 1
    lcd1602_write_cmd(0x01); // 清屏
}

// 清除 LCD 显示
void lcd1602_clear()
{
    lcd1602_write_cmd(0x01); // 清屏命令
}

// 在指定位置显示字符串

```

```

void lcd1602_show_string(u8 x, u8 y, char str[16])
{
    u16 i;

    if (y) x |= 0x40; // 如果 y=1, 则设置第二行
    x |= 0x80; // 设置起始地址

    lcd1602_write_cmd(x); // 写入地址

    for (i = 0; str[i] != '\0' && i < 16; i++) {
        lcd1602_write_dat(str[i]); // 逐字符写入数据
    }
}

#include "timer.h"

// 全局变量定义

float time = 0; // 用于记录时间的变量
float counter = 0; // 用于计数的变量
u16 dutyCycleSum, dutyCycleFirst, dutyCycleSecond, percentageBase; //
用于存储占空比计算的变量

float frequencyRate; // 用于存储频率计算结果的变量

char frequencyStr[16] = "fre:          "; // 用于显示频率的字符串
char dutyCycleStr[16] = "cycle:      %"; // 用于显示占空比的字符串
sbit on = P3 ^ 4; // 定义一个特殊功能位, 用于检测信号

unsigned long int integerPart, decimalPart; // 用于存储频率的整数部分
和小数部分

float decimal; // 用于存储频率的小数部分

float lastFrequencyRate = 0; // 用于存储上一次的频率值

// 初始化定时器的函数

void InitializeTimers() {

    TMOD = 0x15; // 设置定时器模式 T0 计数, T1 计时

```

```

    TH1 = (65536 - 50000) / 256; // 设置定时器 1 的初始值（高 8 位）
    TL1 = (65536 - 50000) % 256; // 设置定时器 1 的初始值（低 8 位）
    TH0 = 0x00; // 设置定时器 0 的初始值（高 8 位）
    TL0 = 0x00; // 设置定时器 0 的初始值（低 8 位）
    ET0 = 1; // 使能定时器 0 中断
    ET1 = 1; // 使能定时器 1 中断
    EA = 1; // 开启全局中断
}

// 定时器 0 的中断服务例程
void Timer0_ISR() interrupt 1 {
    counter++; // 每次中断，计数器加 1
    TH0 = 0; // 重新加载定时器 0 的初始值（高 8 位）
    TL0 = 0; // 重新加载定时器 0 的初始值（低 8 位）
}

// 定时器 1 的中断服务例程
void Timer1_ISR() interrupt 3 {
    time++; // 每次中断，时间计数器加 1
    TH1 = (65536 - 50000) / 256; // 重新加载定时器 1 的初始值（高 8 位）
    TL1 = (65536 - 50000) % 256; // 重新加载定时器 1 的初始值（低 8 位）
}

// 格式化整数部分的函数，用于显示频率
void FormatInteger(unsigned long int intPart, char *frequency) {
    char i;
    for (i = 5; i >= 0; i--) {
        frequency[i + 4] = (intPart % 10) + '0'; // 将整数部分的每一位
        数字转换为字符
        intPart = intPart / 10; // 除以 10，准备下一位数字
    }
}

```

```

    }

    for (i = 4; i < 10; i++) {
        if (frequency[i] == '0') {
            frequency[i] = ' '; // 将前导零替换为空格
        } else {
            break; // 遇到非零数字时停止替换
        }
    }
}

// 计算并显示频率的函数
void CalculateAndDisplayFrequency() {
    while (time < 20); // 等待一定时间

    TR0 = 0; // 停止定时器 0
    TR1 = 0; // 停止定时器 1

    frequencyRate = ((float)counter * 65536.51) + (TH0 * 255.54) + TL0;

    // 计算频率

    TH0 = 0x00; // 重置定时器 0 的值
    TL0 = 0x00;

    TH1 = (65536 - 50000) / 256; // 重置定时器 1 的值
    TL1 = (65536 - 50000) % 256;

    time = 0; // 重置时间计数器
    counter = 0; // 重置计数器

    integerPart = (long int)frequencyRate; // 获取频率的整数部分
    decimal = frequencyRate - integerPart; // 计算频率的小数部分
    decimalPart = (int)(decimal * 100); // 将小数部分转换为整数
    if (frequencyRate != lastFrequencyRate) { // 如果频率发生变化
        lcd1602_show_string(0, 0, "Welcome to use ");
    }
}

```

```

        lcd1602_show_string(0, 1, "Please Wating...");
        delay(2000); // 延时 2 秒显示欢迎等待信息
    }

    FormatInteger(integerPart, frequencyStr); // 格式化整数部分
    frequencyStr[10] = '.'; // 设置小数点
    frequencyStr[11] = decimalPart / 10 + '0'; // 设置小数部分的第一位
    frequencyStr[12] = 'H';
    frequencyStr[13] = 'Z';
    frequencyStr[14] = '\0';

    lcd1602_show_string(0, 0, frequencyStr); // 显示频率
    lastFrequencyRate = frequencyRate; // 更新上一次的频率值
}

// 计算并显示占空比的函数
void CalculateAndDisplayDutyCycle() {
    while (on); // 等待信号变为高电平
    while (!on); // 等待信号变为低电平
    TR1 = 1; // 启动定时器 1
    while (on); // 等待信号再次变为高电平
    TR1 = 0; // 停止定时器 1

    dutyCycleFirst = time * 50000 + (TH1 * 256 + TL1 - 15536); // 计算
    第一个占空比周期

    time = 0; // 重置时间计数器
    TH1 = (65536 - 50000) / 256; // 重置定时器 1 的值
    TL1 = (65536 - 50000) % 256;
    delay(1); // 延时 1 秒
    while (!on); // 等待信号变为高电平
    while (on); // 等待信号变为低电平

```

```

    TR1 = 1; // 启动定时器 1

    while (!on); // 等待信号再次变为低电平

    TR1 = 0; // 停止定时器 1

    dutyCycleSecond = time * 50000 + (TH1 * 256 + TL1 - 15536); // 计算第二个占空比周期

    TH1 = (65536 - 50000) / 256; // 重置定时器 1 的值

    TL1 = (65536 - 50000) % 256;

    time = 0; // 重置时间计数器

    delay(1); // 延时 1 秒

    dutyCycleSum = dutyCycleFirst + dutyCycleSecond; // 计算总周期

    percentageBase = dutyCycleFirst * 100; // 计算基础百分比

    dutyCycleStr[11] = percentageBase / dutyCycleSum / 10 + '0'; // 计算并设置占空比的第一位

    dutyCycleStr[12] = percentageBase / dutyCycleSum % 10 + '0'; // 计算并设置占空比的第二位

    dutyCycleStr[14] = ' '; // 清除多余的字符

    dutyCycleStr[15] = ' '; // 清除多余的字符

    lcd1602_show_string(0, 1, dutyCycleStr); // 显示占空比
}

#include "lcd1602.h"
#include "timer.h"

void main() {
    lcd1602_init();

    InitializeTimers();

    lcd1602_show_string(0, 0, "Welcome to use ");

    lcd1602_show_string(0, 1, "Please Wating...");

    while (1) {

```

```
    TR0 = 1;
    TR1 = 1;
    CalculateAndDisplayFrequency();
    CalculateAndDisplayDutyCycle();
}
}
```